

Divide-and-Conquer

- **Divide** the problem into a number of sub-problems
 - Similar sub-problems of smaller size
- **Conquer** the sub-problems
 - Solve the sub-problems recursively
 - Sub-problem size small enough $\Rightarrow$ solve the problems in straightforward manner
- **Combine** the solutions of the sub-problems
 - Obtain the solution for the original problem

Merge Sort Approach

- To sort an array $A[p \dots r]$:
- **Divide**
 - Divide the n -element sequence to be sorted into two subsequences of $n/2$ elements each
- **Conquer**
 - Sort the subsequences recursively using merge sort
 - When the size of the sequences is 1 there is nothing more to do
- **Combine**
 - Merge the two sorted subsequences

Merge Sort

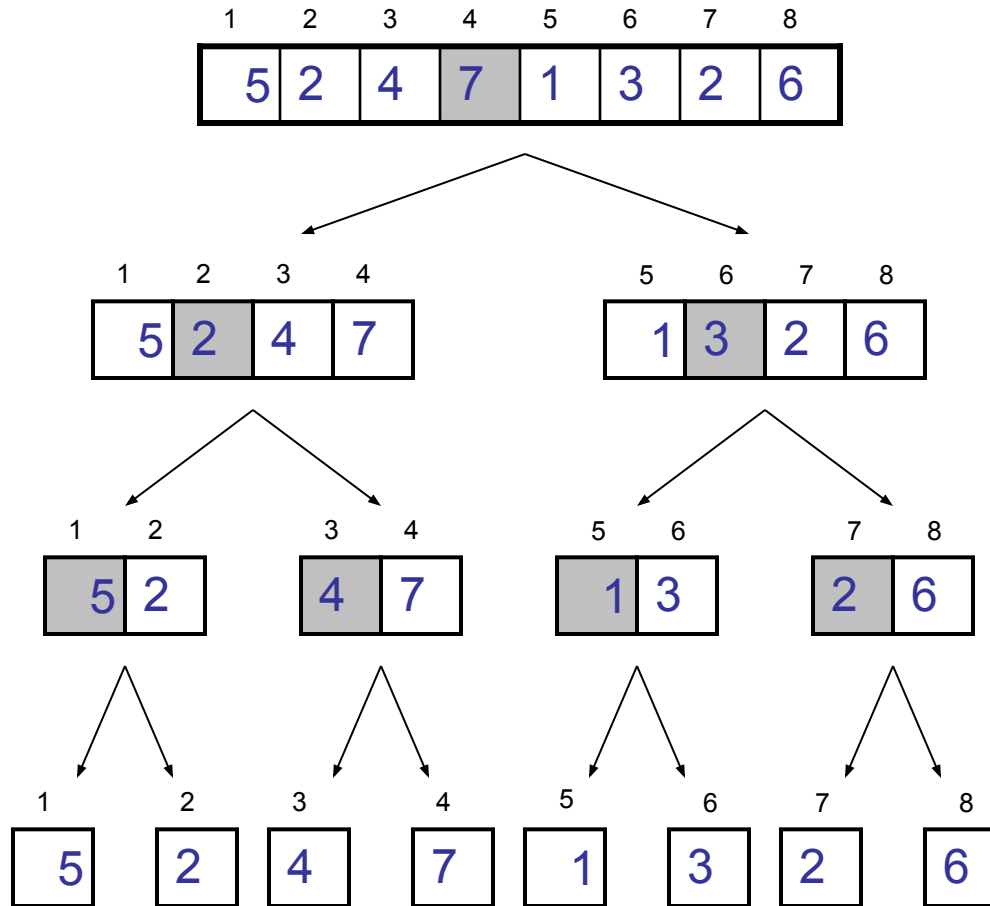
```
1  Algorithm MergeSort(low, high)
2  // a[low : high] is a global array to be sorted.
3  // Small(P) is true if there is only one element
4  // to sort. In this case the list is already sorted.
5  {
6      if (low < high) then // If there are more than one c
7      {
8          // Divide P into subproblems.
9          // Find where to split the set.
10         mid :=  $\lfloor (low + high) / 2 \rfloor$ ;
11         // Solve the subproblems.
12         MergeSort(low, mid);
13         MergeSort(mid + 1, high);
14         // Combine the solutions.
15         Merge(low, mid, high);
16     }
17 }
```

Merging two sorted sub-arrays

```
1  Algorithm Merge(low, mid, high)
2  // a[low : high] is a global array containing two sorted
3  // subsets in a[low : mid] and in a[mid + 1 : high]. The goal
4  // is to merge these two sets into a single set residing
5  // in a[low : high]. b[ ] is an auxiliary global array.
6  {
7      h := low; i := low; j := mid + 1;
8      while ((h ≤ mid) and (j ≤ high)) do
9      {
10         if (a[h] ≤ a[j]) then
11         {
12             b[i] := a[h]; h := h + 1;
13         }
14         else
15         {
16             b[i] := a[j]; j := j + 1;
17         }
18         i := i + 1;
19     }
20     if (h > mid) then
21         for k := j to high do
22         {
23             b[i] := a[k]; i := i + 1;
24         }
25     else
26         for k := h to mid do
27         {
28             b[i] := a[k]; i := i + 1;
29         }
30     for k := low to high do a[k] := b[k];
31 }
```

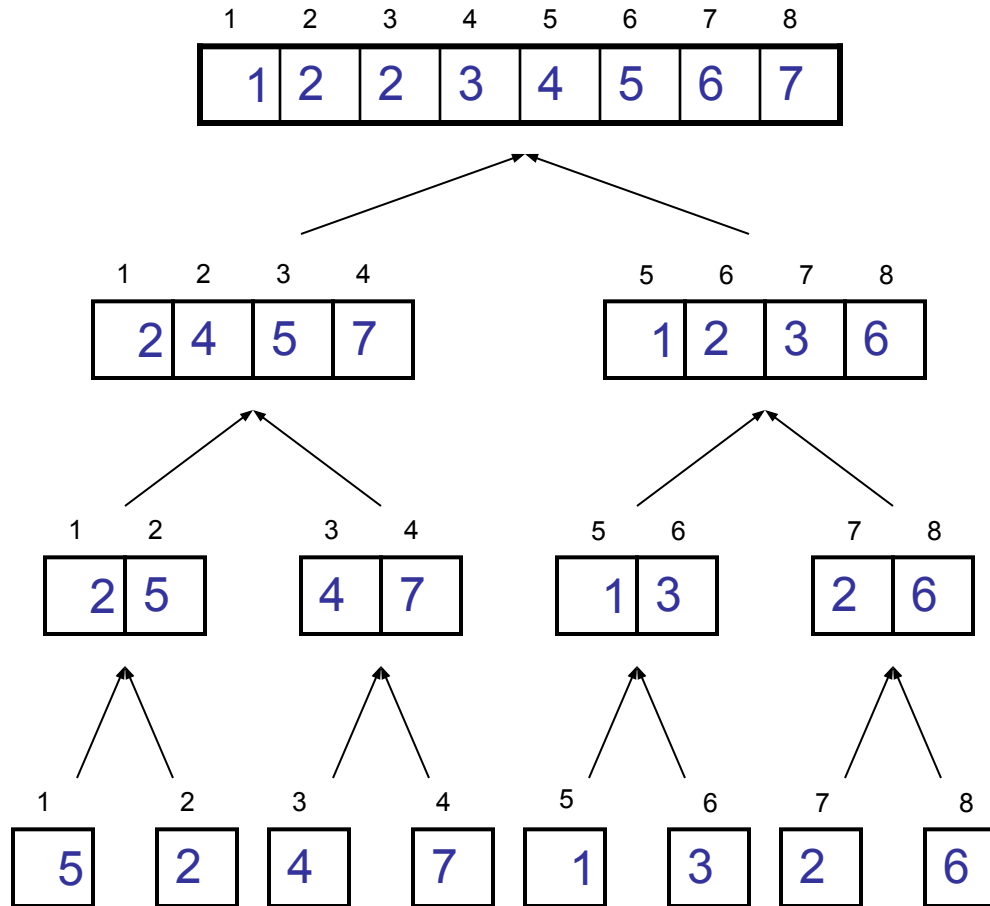
Example – n Power of 2

Divide



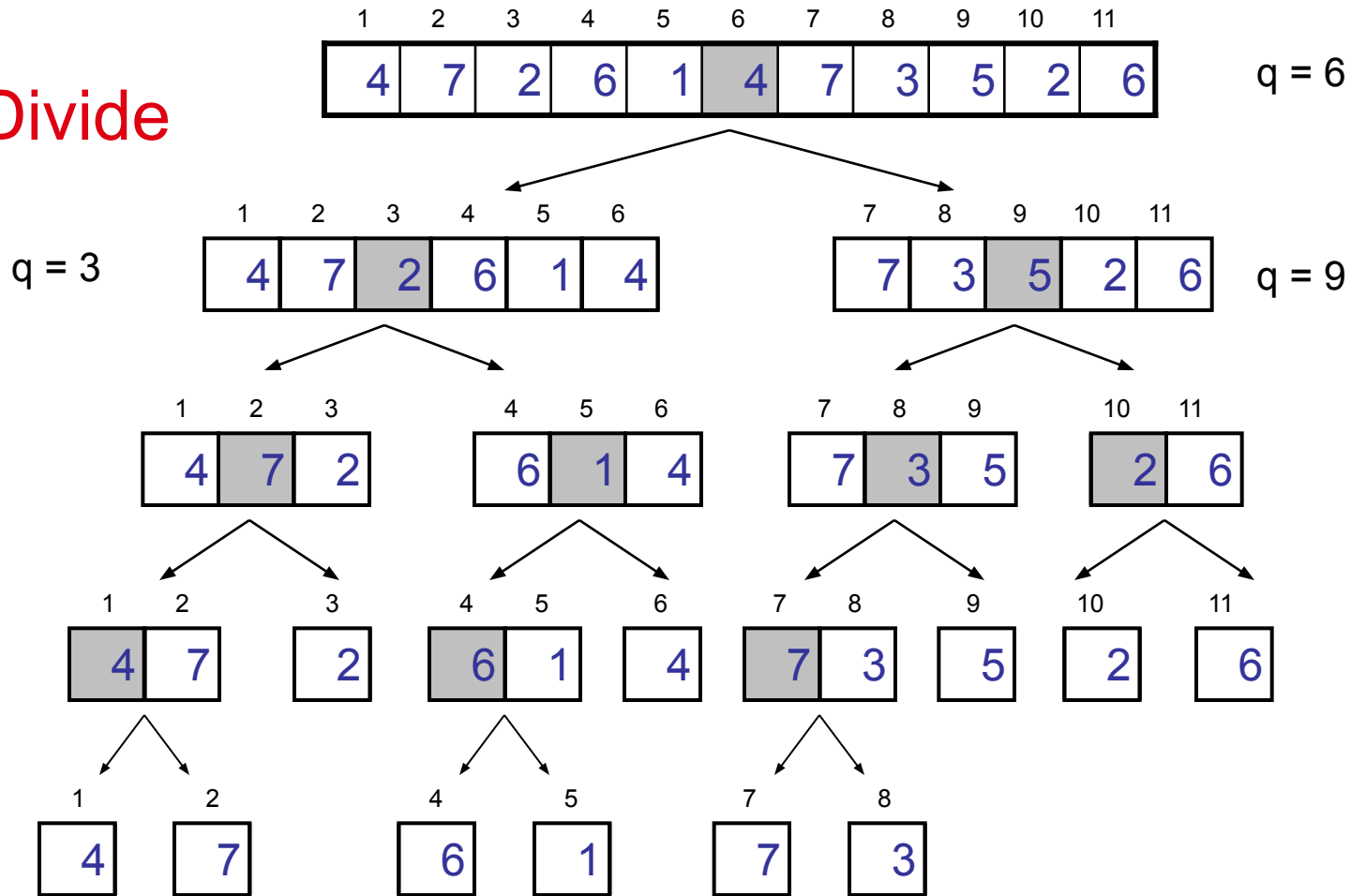
Example – n Power of 2

Conquer
and
Merge



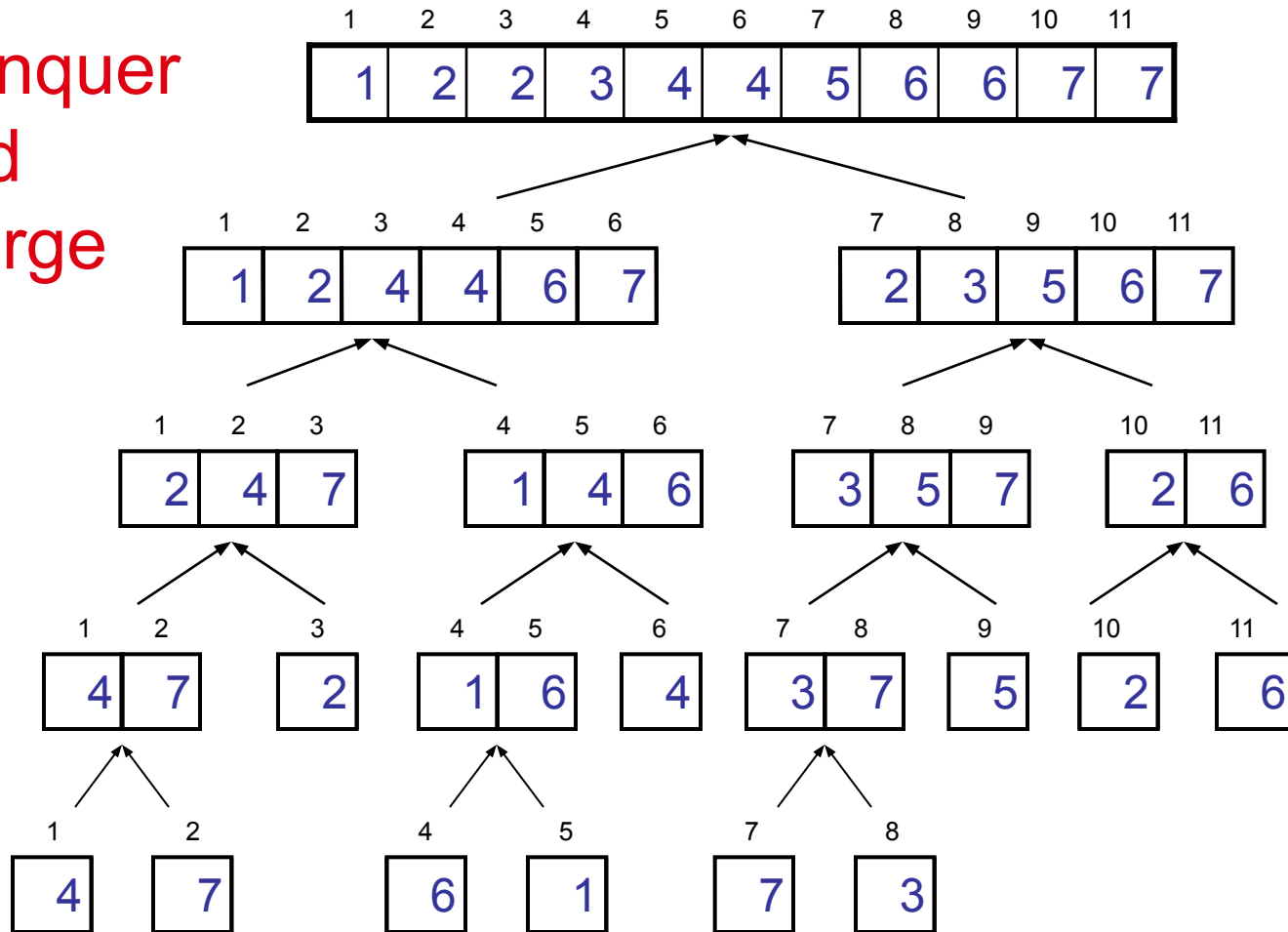
Example – n Not a Power of 2

Divide

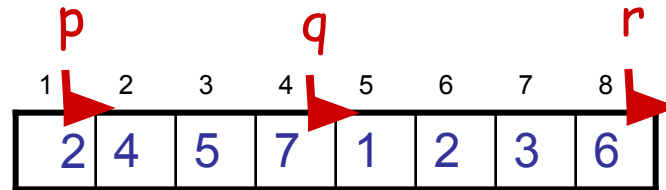


Example – n Not a Power of 2

Conquer
and
Merge



Merging



- **Input:** Array A and indices p, q, r such that $p \leq q < r$
 - Subarrays $A[p \dots q]$ and $A[q + 1 \dots r]$ are sorted
- **Output:** One single sorted subarray $A[p \dots r]$

Merging

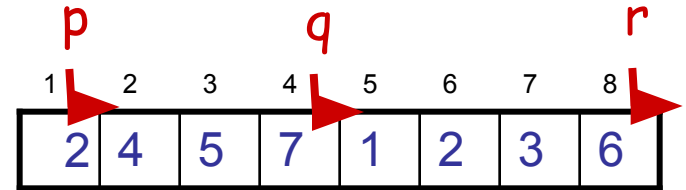
- Idea for merging:

- Two piles of sorted cards

- Choose the smaller of the two top cards
- Remove it and place it in the output pile

- Repeat the process until one pile is empty

- Take the remaining input pile and place it face-down onto the output pile



$A_1 \sqsubset A[p, q]$



$A_2 \sqsubset A[q+1, r]$

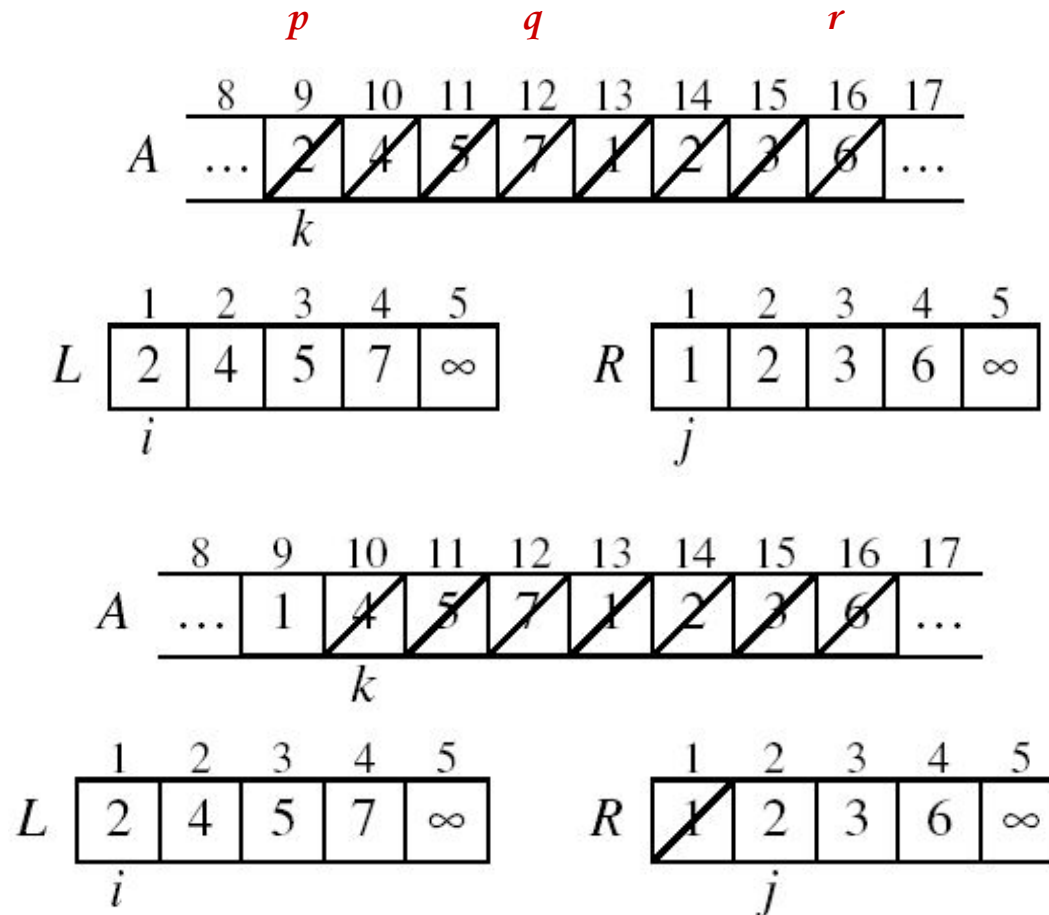


choose the smaller
element from the subarrays

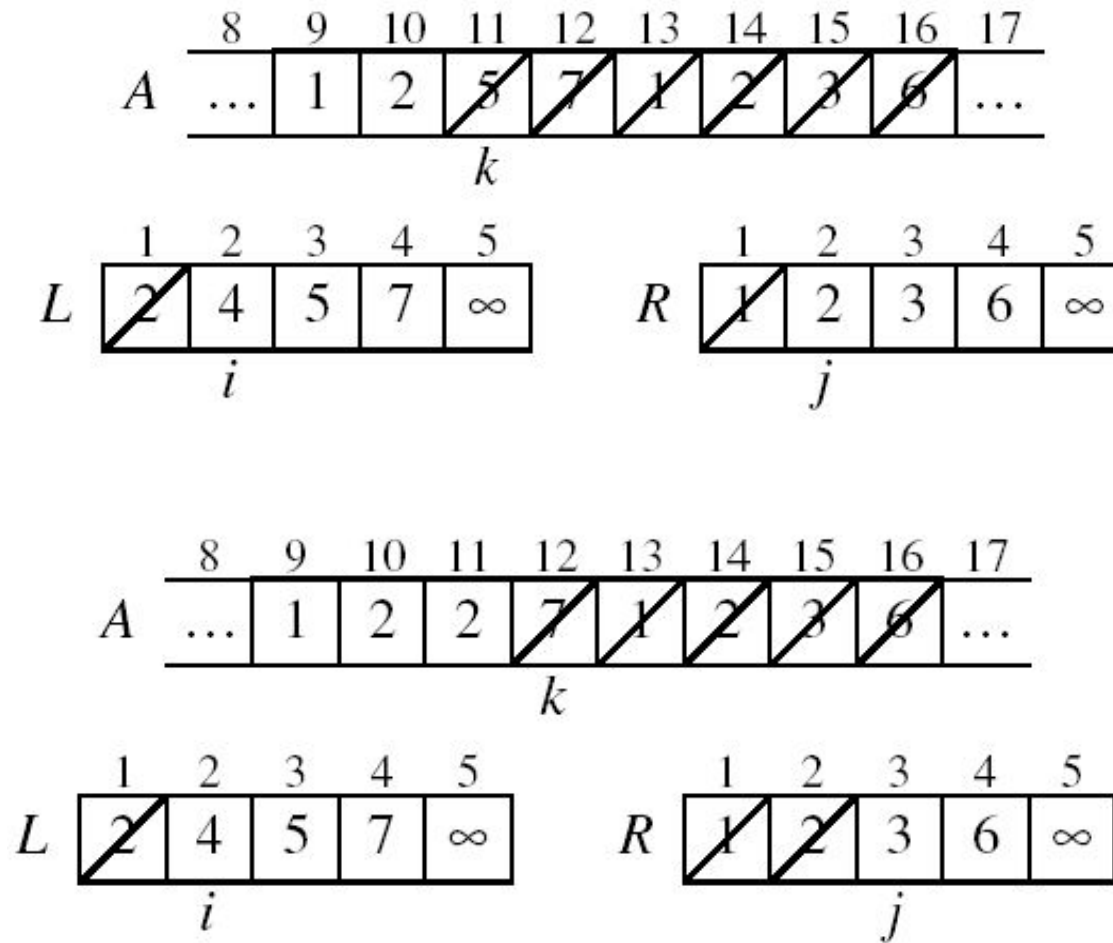
$A[p, r]$



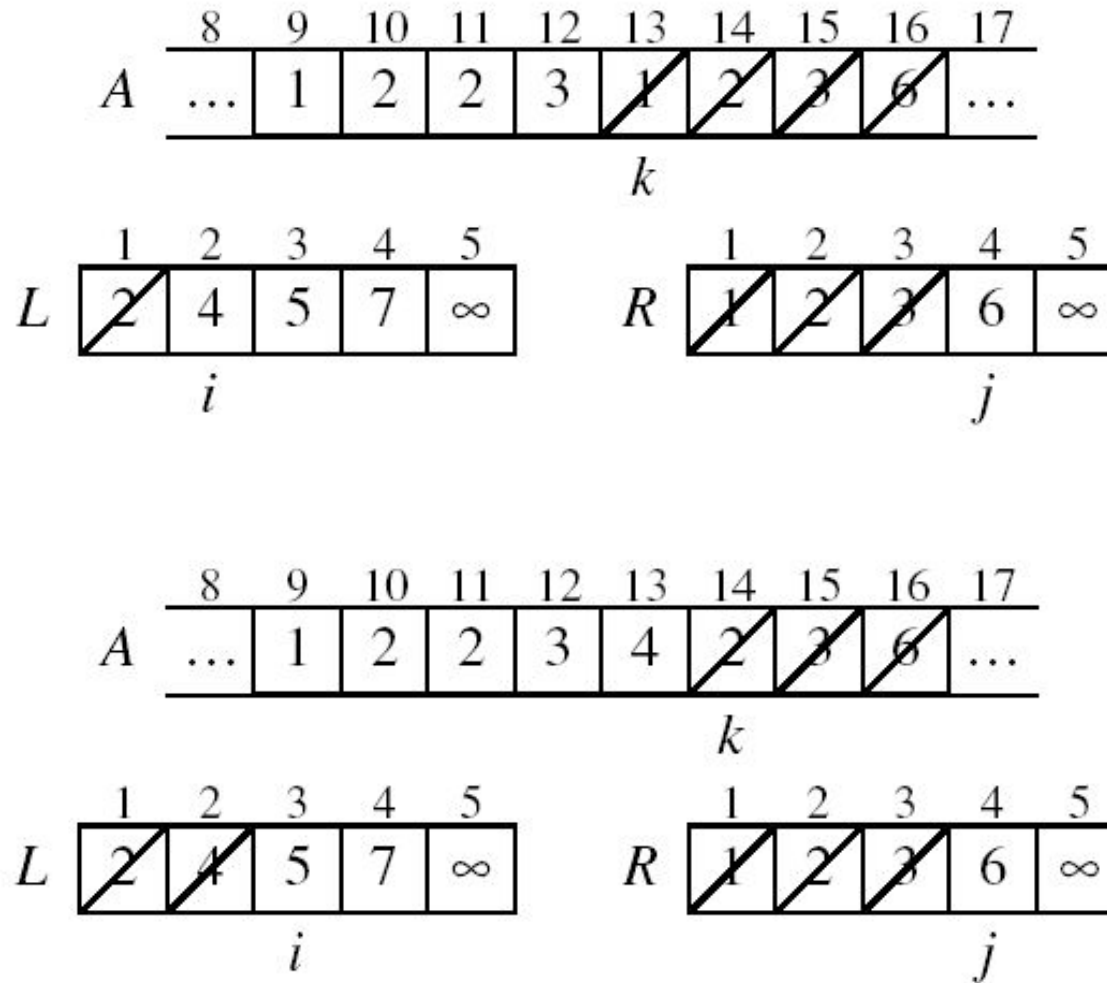
Example: MERGE(A, 9, 12, 16)



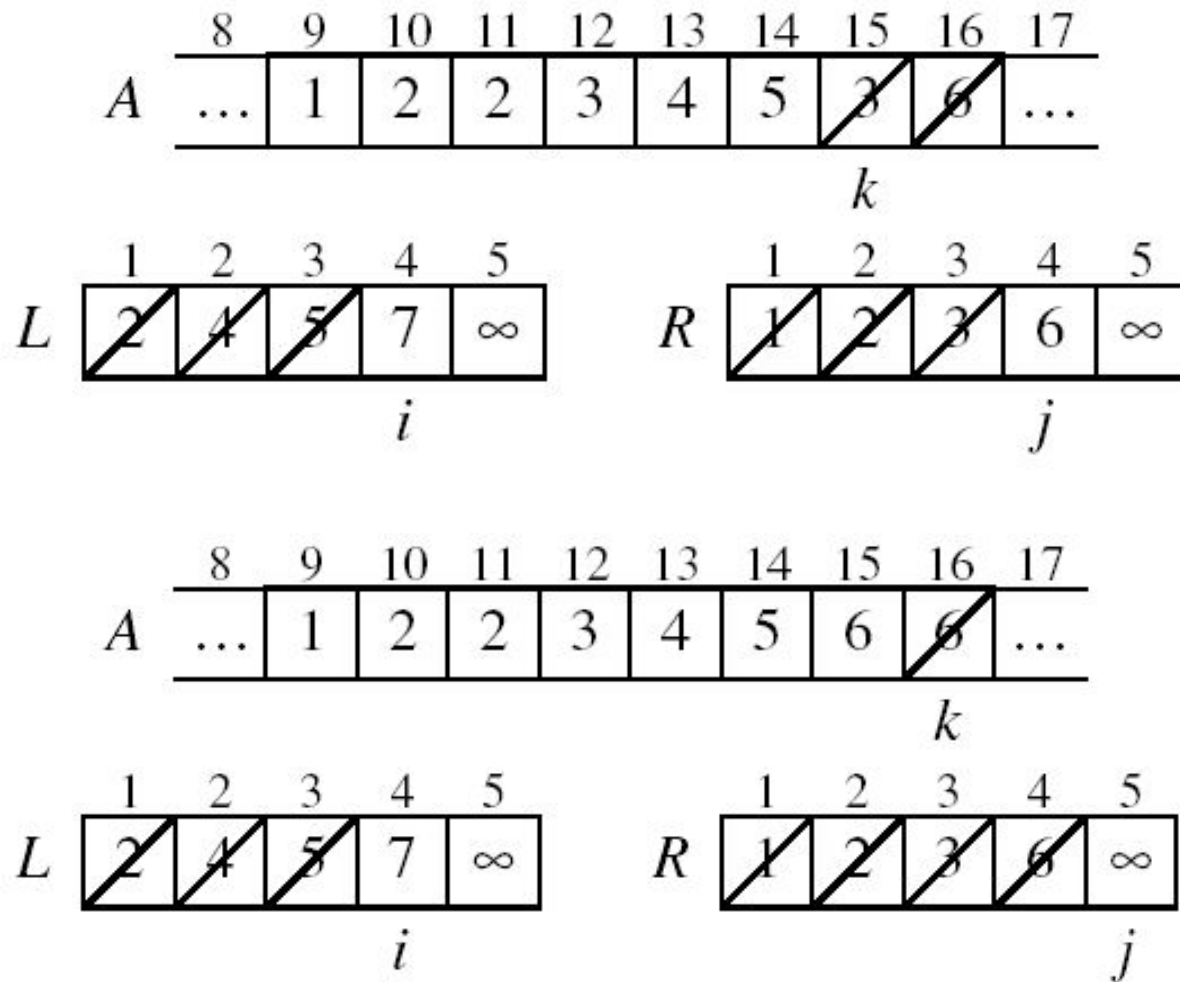
Example: MERGE(A, 9, 12, 16)



Example (cont.)



Example (cont.)



Example (cont.)

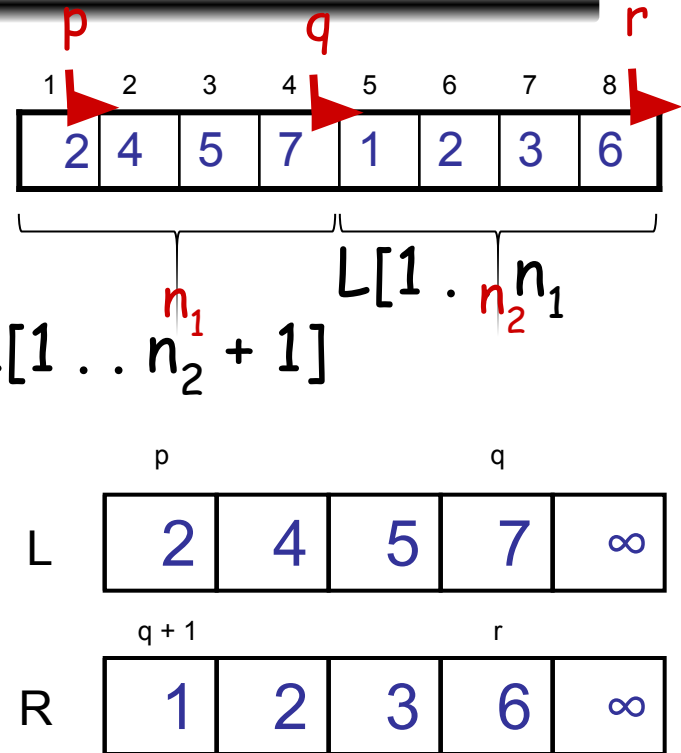
	8	9	10	11	12	13	14	15	16	17	
A	...	1	2	2	3	4	5	6	7	...	
										k	
	1	2	3	4	5						
L	2	4	5	7	∞						
					i						
	1	2	3	4	5						
R	1	2	3	6	∞						
					j						

Done!

Merge - Pseudocode

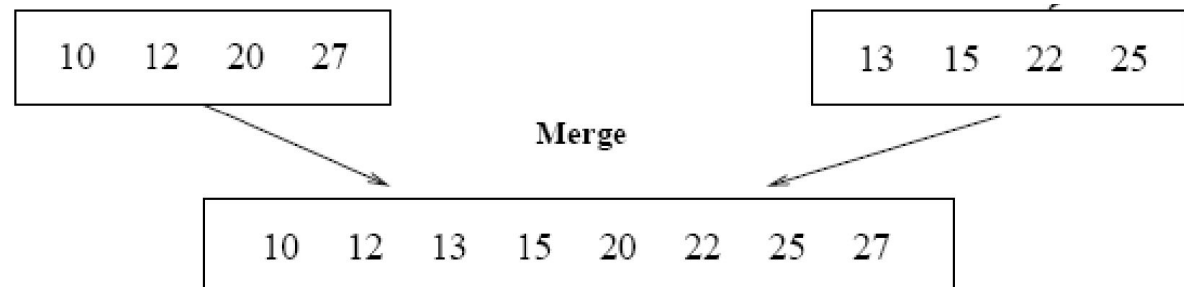
Alg.: MERGE(A, p, q, r)

1. Compute n_1 and n_2
2. Copy the first n_1 elements into $L[1 \dots n_1]$ and the next n_2 elements into $R[1 \dots n_2]$
3. $L[n_1 + 1] \leftarrow \infty$; $R[n_2 + 1] \leftarrow \infty$
4. $i \leftarrow 1$; $j \leftarrow 1$
5. **for** $k \leftarrow p$ **to** r
6. **do if** $L[i] \leq R[j]$
7. **then** $A[k] \leftarrow L[i]$
8. $i \leftarrow i + 1$
9. **else** $A[k] \leftarrow R[j]$
10. $j \leftarrow j + 1$



Running Time of Merge (assume last **for** loop)

- Initialization (copying into temporary arrays):
 - $\Theta(n_1 + n_2) = \Theta(n)$
- Adding the elements to the final array:
 - n iterations, each taking constant time $\Rightarrow \Theta(n)$
- Total time for Merge:
 - $\Theta(n)$



MERGE-SORT Running Time

- **Divide:**

- compute q as the average of p and r : $D(n) = \Theta(1)$

- **Conquer:**

- recursively solve 2 subproblems, each of size $n/2$
 $\Rightarrow 2T(n/2)$

- **Combine:**

- MERGE on an n -element subarray takes $\Theta(n)$ time
 $\Rightarrow C(n) = \Theta(n)$

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + (n) & \text{if } n > 1 \end{cases}$$

Solve the Recurrence

$$T(n) = \begin{cases} c & \text{if } n = 1 \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$

Use Master's Theorem:

Compare n with $f(n) = cn$

Case 2: $T(n) = \Theta(n \lg n)$